Detection Engineering with YARA

Blue Team Series

For brevity and simplicity, we will be examining metasploit payloads and testing to see how concrete we can make our detections.

The simple workflow is:

1. Use FLOSS to find strings
2. Use a hex editor to identify the opcodes responsible for said instructions
3. Write a YARA rule to detect threats based on these signatures

We will use five generated payloads with msfvenom, with the goal of creating a detection that works across several different samples. If this is possible with simple static analysis, it would be a great showcase for the value of more advanced techniques like reverse engineering and analyzing disassembly to make better detections.

Displayed below are the names of the five samples:

There are…

Two binds, two reverse callbacks, and a calc.exe to spawn a calc instance

Name	Date modified	Type	Size
bind	10/13/2024 10:24 AM	Application	7 KB
calc	10/13/2024 10:25 AM	Application	7 KB
https	10/13/2024 10:25 AM	Application	204 KB
meterbind	10/13/2024 10:25 AM	Application	7 KB
tcp	10/13/2024 10:25 AM	Application	7 KB

We will begin by running floss against “tcp.exe”

```
C:\Users\[redacted]\Desktop\Tools>floss DetectionEngineering
INFO: floss: extracting static strings
finding decoding function features: 100%|
INFO: floss.stackstrings: extracting stackstrings from 2 fu
extracting stackstrings: 100%|
INFO: floss.tightstrings: extracting tightstrings from 0 fu
extracting tightstrings: 0 functions [00:00, ? functions/s]
INFO: floss.string_decoder: decoding strings
INFO: floss.results: ws232
emulating function 0x1400040d9 (call 1/1): 100%|
INFO: floss: finished execution after 4.03 seconds
INFO: floss: rendering results


FLARE FLOSS RESULTS (version v3.1.1-0-g3cd3ee6)

+------------------------+[redacted]
| file path              |
| identified language    | unknown
| extracted strings      |
|  static strings        | 24 (206 characters)
|   language strings     |  0 (  0 characters)
|  stack strings         | 0
|  tight strings         | 0
|  decoded strings       | 1
+------------------------+--------------------------------


 ___________________________
  FLOSS STATIC STRINGS (24)
 ___________________________


+---------------------------------+
| FLOSS STATIC STRINGS: ASCII (24) |
+---------------------------------+

!This program cannot be run in DOS mode.
Rich}E
.text
`.rdata
@.lnkl
PAYLOAD:
ExitProcess
VirtualAlloc
KERNEL32.dll
AQAPRH1
QVeH
JJM1
rPH1
RAQH
AX^YZAXAYAZH
XAYZH
ws2_32
EATI
YPPM1
unMa
j@AYh
KERNEL32.dll
VirtualAlloc
ExitProcess
```

We see a string for “PAYLOAD” in the static strings of FLOSS.

We can open a hex editor (HxD) to verify the bytes responsible for this…

```
000003C0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
000003D0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
000003E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
000003F0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00000400  48 83 EC 28 49 C7 C1 40 00 00 00 49 C7 C0 00 30  Hƒì(IÇÁ@...IÇÀ.0
00000410  00 00 48 C7 C2 00 10 00 00 48 33 C9 E8 27 10 00  ..HÇÂ....H3Éè'..
00000420  00 48 C7 C1 00 10 00 00 48 BE 41 10 00 40 01 00  .HÇÁ....H¾A..@..
00000430  00 00 48 8B F8 F3 A4 FF D0 48 33 C9 E8 01 10 00  ..H‹øó¤ÿÐH3Éè...
00000440  00 50 41 59 4C 4F 41 44 3A 00 00 00 00 00 00 00  .PAYLOAD:.......
00000450  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00000460  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00000470  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Then we can create a yara rule!

```
rule Metasploit_Default_Payload
{
    meta:
        description = "YARA rule to detect a Metasploit TCP reverse shell payload based on a suspicious string"
        author = "0xPiLLz"
        date = "2024-10-13"
        reference = "Metasploit TCP reverse shell"

    strings:
        $msf_payload_sig = { 50 41 59 4C 4F 41 44 } //"PAYLOAD in hex

    condition:
        $msf_payload_sig
}
```

When testing this rule against the other samples we get a match for all five!

```
C:\Users\[redacted]\Desktop\Tools\yara>yara64.exe metasploit.yar c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\tcp.exe
Metasploit_Default_Payload c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\tcp.exe

C:\Users\[redacted]\Desktop\Tools\yara>yara64.exe metasploit.yar c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\calc.exe
Metasploit_Default_Payload c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\calc.exe

C:\Users\[redacted]\Desktop\Tools\yara>yara64.exe metasploit.yar c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\bind.exe
Metasploit_Default_Payload c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\bind.exe

C:\Users\[redacted]\Desktop\Tools\yara>yara64.exe metasploit.yar c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\meterbind.exe
Metasploit_Default_Payload c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\meterbind.exe

C:\Users\[redacted]\Desktop\Tools\yara>yara64.exe metasploit.yar c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\https.exe
Metasploit_Default_Payload c:\Users\[redacted]\Desktop\Tools\DetectionEngineering\https.exe
```

Likely, this was a purposeful indicator of compromise thrown in by the developers at Rapid7 to deter script kiddies and allow for easy detection. That being said, it is likely easily circumvented with little effort from an experienced adversary.

Let's continue our analysis…
There is consistency across the first four samples (excluding the reverse https meterpreter payload), being the PAYLOAD string, the DOS stub, and a limited amount of imports.

```
!This program cannot be run in DOS mode.
Rich}E
.text
`.rdata
@.fece
PAYLOAD:
ExitProcess
VirtualAlloc
KERNEL32.dll
AQAPRQVH1
JJM1
RAQH
AXAX^YZAXAYAZH
XAYZH
ws2_32
FATI
YPPM1
unMa
j@AYh
KERNEL32.dll
VirtualAlloc
ExitProcess
```

KERNEL32.dll is imported into the address space of any third party program, and most any other for that matter, by default. It is the primary library making up the WinAPI and allows transition between user and kernel mode to perform tasks specified by the program. I have written on this more in depth in other posts.

VirtualAlloc is an API function contained within Kernel32 that may be used to allocate memory and is commonly used in simple shellcode loaders. However, it is absolutely not an indicator of compromise on its own, as pretty much any process you could think of needs to allocate memory to perform operations at some point.

The most interesting string within this entire snippet is the “ws2_32” string. This is indicative of a 32-bit DLL used to enable socket manipulations:

“Ws2_32.dll loads the service provider's interface DLL [via] Microsoft Windows dynamic library loading mechanisms, and initializes it by calling WSPStartup. This is typically triggered by an application calling either socket or WSASocket to create a new socket to be associated with a service provider whose interface DLL is not currently loaded into memory”

Initialization - Win32 apps | Microsoft Learn
Windows Sockets 2 - Win32 apps | Microsoft Learn

This could be an excellent resource for debugging and making more concrete rules…